大数据隐私保护技术实验报告

姓名：姚嫣如　　学号：21080012078　　实验日期：2026.09.12

一、实验要求

- 数据集中每行表示一位用户的交易记录，商品编号以空格分隔，不同用户的交易长度可以不同。
- 统计并发布 `item1` 到 `itemd` 的购买人次，即所有商品的频数分布。
- 在有可信第三方的场景下，使用中心化差分隐私的拉普拉斯机制扰动统计结果。
- 在无可信第三方的场景下，每位用户先随机抽取一个商品，再使用随机响应或 UE 机制在本地扰动。本实验选择 UE 机制。
- 分别取隐私预算 `ϵ=0.1、1、2、4`，使用平均平方误差（MSE）比较两种场景的准确度，并给出结果表和曲线图。

二、实验思路

- **生成模拟数据**：由于没有给出数据集，先生成 10,000 条模拟交易。商品域大小设为 100，交易长度服从截断的泊松分布，商品热度采用 Zipf 长尾分布，并加入简单的类别偏好。
- **可信第三方**：接收完整交易，统计真实频数后加入拉普拉斯噪声。单个用户的商品贡献上限设为 20，据此确定噪声尺度。
- **无可信第三方**：每位用户从自己的交易中等概率抽取一个商品，将它转成长度为 100 的一元编码，再按 UE 概率逐位扰动。聚合端根据收到的比特向量反推商品频数。
- **重复实验获取结果**：每个 `ϵ` 重复实验 50 次。频数估计值被限制在合法区间内，最后统计 MSE 的均值和标准差，并用对数纵轴绘制曲线。

三、实验分析及结果

1、 模拟数据说明

模拟数据不是简单的均匀随机数。现实交易中通常有少数热门商品和大量低频商品，因此生成时使用 Zipf 分布控制总体热度，再随机给每位用户分配偏好类别。这样生成的频数有较明显的长尾特征，也会出现同类商品共同进入交易的情况。

数据项	数值
用户数	10,000
商品种类数 `d`	100
商品编号范围	1 ~ 100

数据项	数值
商品出现总次数	54,824
实际最短交易长度	1
实际最长交易长度	15
平均交易长度	5.4824
规定的单用户贡献上限	20

数据中每条交易只保留不同的商品编号，所以同一位用户不会对同一商品重复计数。贡献上限 20 是实验开始前规定的约束。虽然本次模拟数据中观察到的最大长度是 15，敏感度仍按 20 计算，不能根据当前样本临时缩小。

2、 方案设计

方案一：可信第三方下的拉普拉斯机制

1. 可信第三方收集完整交易，检查商品编号、重复商品和交易长度是否符合约束。
2. 统计 100 种商品的精确购买人次，得到频数向量 $q(D)$。
3. 根据敏感度 20 和当前隐私预算 `ε`，为每个频数加入 $Lap(20/\varepsilon)$ 噪声。
4. 将带噪频数限制在 `[0, 10000]` 内并发布，再与精确频数比较，计算 MSE。

方案二：无可信第三方下的 UE 机制

1. 每位用户在本地从自己的交易记录中等概率抽取一个商品，并生成长度为 100 的 one-hot 向量。
2. 用户根据当前 `ε` 计算 p 和 q，对向量中的每个比特独立扰动，只上传扰动结果。
3. 聚合端按位统计 1 的数量，通过 $(I_i - nq)/(p-q)$ 估计每种商品的频数。
4. 对估计值做合法区间截断，与抽样后的精确频数比较，计算 MSE。

3、 敏感度分析

（1）可信第三方场景

设完整的频数查询为：

$$q(D) = (f_1, f_2, \dots, f_d)$$

本实验采用 add/remove 用户级邻接关系，即相邻数据集相差一位用户。每位用户最多包含 20 个不同商品，加入或删除该用户时，最多有 20 个频数各变化 1。因此，频数向量的 L1 全局敏感度为：

$$\Delta_1 q = \max_{D \sim D'} \|q(D) - q(D')\|_1 = 20$$

拉普拉斯机制对每个频数加入独立噪声：

$$\tilde{f}_i = f_i + Lap\left(\frac{20}{\varepsilon}\right)$$

拉普拉斯变量的方差是 $2b^2$。不考虑合法区间截断时，各 `ε` 对应的噪声尺度和理论 MSE 如下。

`ε`	**噪声尺度 `b=20/ε`**	**理论 MSE `2b²`**
0.1	200	80,000
1	20	800
2	10	200
4	5	50

（2）无可信第三方场景

每位用户先随机抽取一个商品，再将商品编号写成 one-hot 向量。UE 对每一位分别扰动，参数为：

$$p = \frac{e^{\varepsilon/2}}{e^{\varepsilon/2}+1}, \qquad q = \frac{1}{e^{\varepsilon/2}+1}$$

原比特为 1 时，以概率 p 报告 1；原比特为 0 时，以概率 q 报告 1。两个不同商品的一元编码有两个位置不同，按上述参数计算，任意输出在两个输入下的概率比不超过 e^{ε}，满足 `ε-LDP`。

假设共有 n 位用户，商品 i 的抽样真实频数为 f_i，聚合后在第 i 位收到的 1 的数量为 I_i，则：

$$E[I_i] = f_i p + (n - f_i) q$$

由此得到无偏估计：

$$\hat{f}_i = \frac{I_i - nq}{p - q}$$

这里的 UE 误差以“抽样后、扰动前”的精确频数为基准。抽样结果在所有 `ε` 和重复轮次中保持不变，这样算出的 MSE 主要来自隐私扰动，不把每次重新抽样的随机波动混在里面。

4、 评价指标

两种场景都使用平均平方误差：

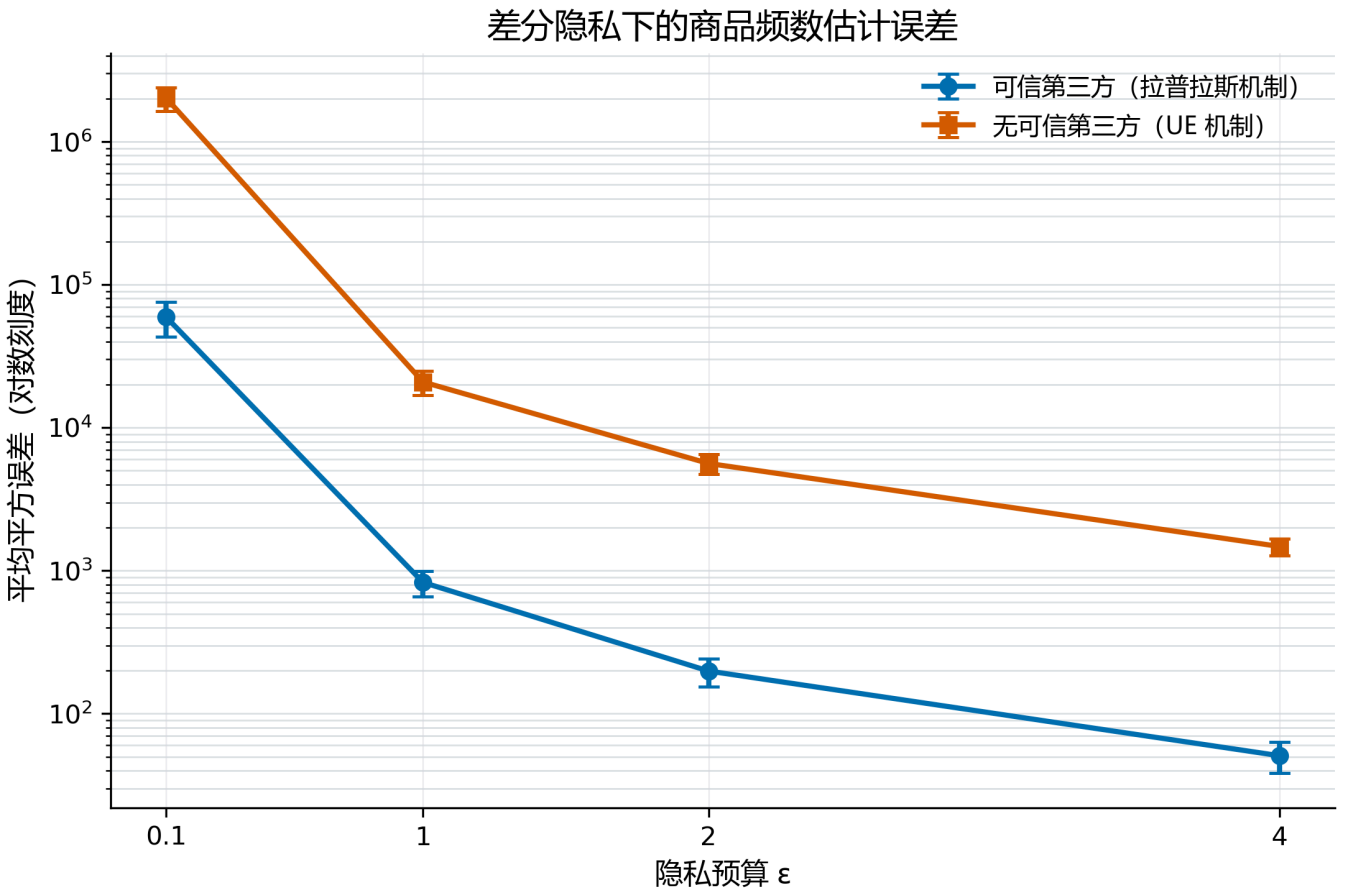
$$MSE = \frac{1}{d}\sum_{i=1}^{d}(f_i - \hat{f}_i)^2$$

其中，$d = 100$，f_i 是无噪声频数，$\hat{f}_i$ 是发布的带噪频数。MSE 越小，发布结果越接近精确统计值。

5、 实验结果比对

每个隐私预算独立重复 50 次。下表给出 MSE 均值和标准差，最后一列用于观察两种场景的误差差距。

隐私预算 ϵ	可信第三方：Laplace	无可信第三方：UE	UE/Laplace MSE 比值
0.1	59,414.69 ± 16,342.15	2,021,501.63 ± 378,640.86	34.02
1	827.20 ± 167.45	20,865.65 ± 3,963.00	25.22
2	198.48 ± 44.68	5,619.05 ± 889.88	28.31
4	50.74 ± 12.56	1,475.59 ± 197.96	29.08



曲线很直观：ϵ 增大后，两种机制的 MSE 都持续下降。以 ϵ=0.1 为例，UE 的平均 MSE 超过 202 万；到 ϵ=4 时降到 1,475.59。拉普拉斯机制也有相同趋势，平均 MSE 从 59,414.69 降到 50.74。

四组结果中，UE 的误差约为拉普拉斯机制的 25 ~ 34 倍。原因在于两种场景的信任条件不同。可信第三方可以先看到精确频数，只需在最终结果上加一次噪声；UE 则要求每位用户先把自己的 100 位编码扰动，聚合端还要除以较小的 $p - q$ 来还原频数。当 ϵ 很小时，p 和 q 十分接近，估计误差会被放大。

ϵ=0.1 时，拉普拉斯机制的实验 MSE 低于未截断的理论值 80,000。这不是公式失效，而是负频数被截断为 0 后，一部分极端误差被削弱。ϵ=2 和 ϵ=4 时噪声较小，实验值已经接近理论值。

6、 实验代码

下面列出实验中使用的核心实现。

模拟数据生成核心代码

```python
import math
import random


def poisson_sample(mean, rng):
    threshold = math.exp(-mean)
    product = 1.0
    count = 0
    while product > threshold:
        count += 1
        product *= rng.random()
    return count - 1


def generate_transactions(users=10000, items=100, seed=20260914):
    rng = random.Random(seed)
    base_weights = [1.0 / rank**1.1 for rank in range(1, items + 1)]
    transactions = []

    for _ in range(users):
        # 交易长度为 1+Poisson(4.5)，并限制在 20 以内。
        length = min(1 + poisson_sample(4.5, rng), 20)
        preferred_category = rng.randrange(10)

        # 用户偏好类别中的商品权重扩大 4 倍。
        weights = []
        for index, base_weight in enumerate(base_weights):
            category = min(index * 10 // items, 9)
            boost = 4.0 if category == preferred_category else 1.0
            weights.append(base_weight * boost)

        # 使用指数键完成加权无放回抽样，同一交易不会出现重复商品。
        keys = [
            (-math.log(max(rng.random(), 1e-15)) / weight, index + 1)
            for index, weight in enumerate(weights)
        ]
        keys.sort()
        transaction = sorted(item_id for _, item_id in keys[:length])
        transactions.append(transaction)

    return transactions
```

差分隐私机制与实验主循环

```python
import math
import numpy as np

# 统计完整交易中每种商品的精确购买人次
def full_frequency(transactions, domain_size):
    counts = np.zeros(domain_size, dtype=np.float64)
```

```python
    for transaction in transactions:
        counts += np.bincount(transaction - 1, minlength=domain_size)
    return counts

# 每位用户从自己的交易中等概率抽取一个商品
def sample_one_item_per_user(transactions, rng):
    return np.asarray(
        [transaction[rng.integers(len(transaction))] for transaction in transactions]
    )


def mse(exact, estimate):
    return float(np.mean((exact - estimate) ** 2))

# 可信第三方场景：在最终频数上加入拉普拉斯噪声
def laplace_estimate(exact, epsilon, sensitivity, rng, users):
    scale = sensitivity / epsilon
    noisy = exact + rng.laplace(0.0, scale, size=exact.shape)
    return np.clip(noisy, 0.0, users)


def ue_parameters(epsilon):
    exp_half = math.exp(epsilon / 2.0)
    p = exp_half / (exp_half + 1.0)
    q = 1.0 / (exp_half + 1.0)
    return p, q

# 将商品编号转换为 one-hot 向量
def unary_encode(sampled_items, domain_size):
    encoded = np.zeros((len(sampled_items), domain_size), dtype=np.uint8)
    encoded[np.arange(len(sampled_items)), sampled_items - 1] = 1
    return encoded

# 用户本地扰动编码，聚合端再进行无偏频数估计
def ue_estimate(encoded, epsilon, rng):
    p, q = ue_parameters(epsilon)
    probabilities = np.where(encoded == 1, p, q)
    reports = rng.random(encoded.shape) < probabilities
    reported_ones = reports.sum(axis=0)
    estimate = (reported_ones - len(encoded) * q) / (p - q)
    return np.clip(estimate, 0.0, len(encoded))


def run_experiment(transactions):
    epsilons = [0.1, 1.0, 2.0, 4.0]
    repetitions = 50
    domain_size = 100
    sensitivity = 20
    users = len(transactions)
    rng = np.random.default_rng(20260914)

    # 两种场景使用各自的无噪声频数作为 MSE 基准
    exact_full = full_frequency(transactions, domain_size)
```

```
    sampled_items = sample_one_item_per_user(transactions, rng)
    exact_sampled = np.bincount(
        sampled_items - 1, minlength=domain_size
    ).astype(np.float64)
    encoded = unary_encode(sampled_items, domain_size)

    results = []
    for epsilon in epsilons:
        for repetition in range(1, repetitions + 1):
            central = laplace_estimate(
                exact_full, epsilon, sensitivity, rng, users
            )
            local = ue_estimate(encoded, epsilon, rng)
            results.append(
                {
                    "epsilon": epsilon,
                    "repetition": repetition,
                    "laplace_mse": mse(exact_full, central),
                    "ue_mse": mse(exact_sampled, local),
                }
            )
    return results
```

MSE 曲线绘制代码

```
import matplotlib.pyplot as plt


figure, axis = plt.subplots(figsize=(7.2, 4.8), constrained_layout=True)

axis.errorbar(
    epsilons,
    laplace_mean_mse,
    yerr=laplace_std_mse,
    label="可信第三方（拉普拉斯机制）",
    color="#0072B2",
    marker="o",
    linewidth=2,
    capsize=4,
)
axis.errorbar(
    epsilons,
    ue_mean_mse,
    yerr=ue_std_mse,
    label="无可信第三方（UE 机制）",
    color="#D55E00",
    marker="s",
    linewidth=2,
    capsize=4,
)

axis.set_yscale("log")
```

```
axis.set_xlabel("隐私预算 ε")
axis.set_ylabel("平均平方误差（对数刻度）")
axis.set_title("差分隐私下的商品频数估计误差")
axis.legend(frameon=False)
axis.grid(axis="y", which="both", alpha=0.4)
figure.savefig("results/mse_comparison.png", dpi=300)
```

四、实验心得

通过本次实验，我对差分隐私和本地化差分隐私的基本思想有了更直观的理解。实验中，我分别采用拉普拉斯机制和随机响应、UE 等方法对商品购买频数进行隐私保护，并通过不同隐私预算 ε 下的 MSE 比较数据可用性。

实验结果让我认识到，隐私保护强度与数据精度之间存在明显权衡：ε 较小时隐私保护更强，但加入的扰动更大，统计误差也会增加；ε 较大时则能获得更准确的统计结果。同时，通过对可信第三方和无可信第三方两种场景的比较，我进一步理解了中心化差分隐私与本地化差分隐私在数据处理流程和隐私保护位置上的区别。本次实验也加深了我对敏感度、噪声机制和频数估计等概念的理解。